

AI Automation Platform

Building n8n, from an empty folder

The complete build guide — second edition. Every listing in this document is read out of the repository at build time, so the code on these pages is the code that runs.



13

node types

142

automated tests

14

HTTP routes

8

people, 8 areas

Schedule trigger · Webhook · HTTP · Google Docs, Sheets, Drive, Gmail · Code node · AI
Demonstration week 14 · Report week 15

1 What changed in this edition

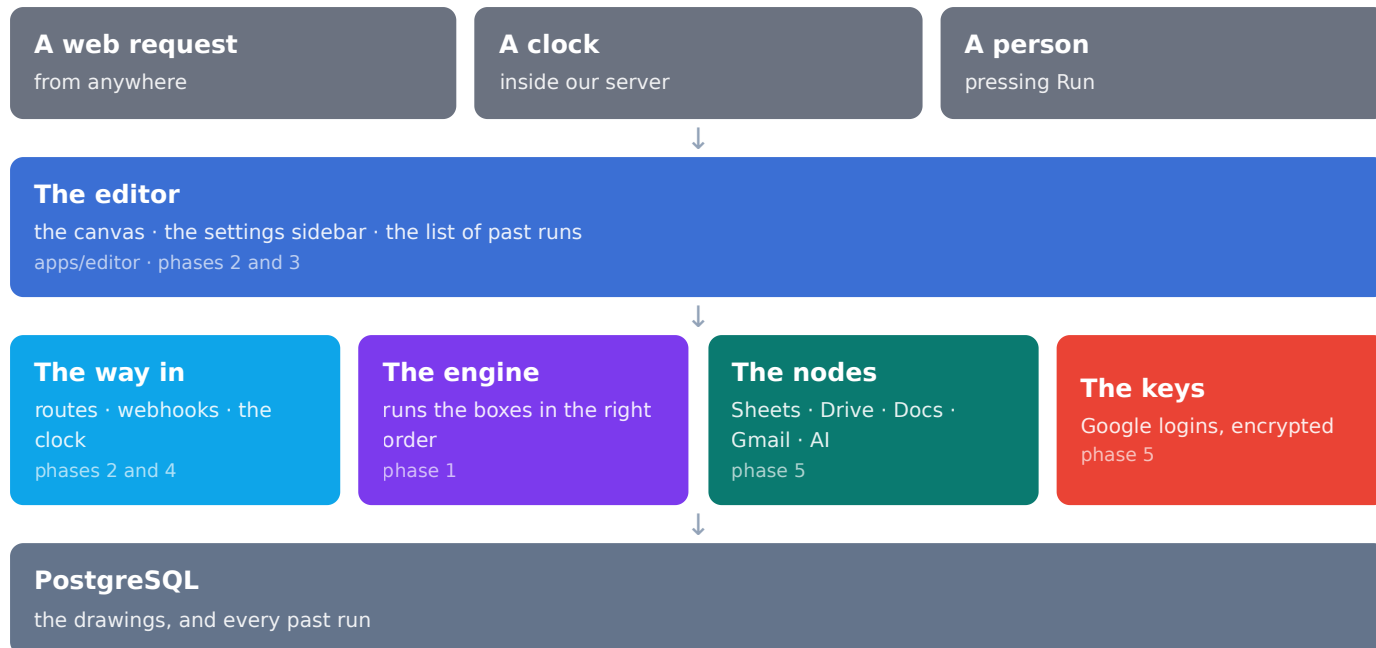
The first edition was written before the code existed. Building it turned up seven places where the design on the page did not survive contact with a running system. Every one of them is a change a marker can ask about, so they are listed here rather than quietly folded in.

WAS	IS NOW	WHY
A ready-list run loop	Topological order (ADR 0002)	A ready list runs Merge twice , once per incoming line, each time with half its data. Sorting first runs every node exactly once.
A loop is caught by the step limit	A loop is refused by <code>validateWorkflow</code> , naming the nodes on it	A cycle has no running order at all. <code>maxSteps</code> stays as a second belt.
<code>new Function(..., 'eval', ...)</code> shadows <code>eval</code>	<code>eval</code> and <code>arguments</code> are refused outright	Strict mode forbids binding either name — the original line is a <code>SyntaxError</code> , so no expression compiled.
Code node runs <code>vm.runInNewContext(code)</code>	The program is wrapped in <code>(async function(){...})()</code>	Everybody writes <code>return items;</code> , and a top-level return outside a function is illegal.
<code>logs.push(...).slice(0,100)</code>	A length check before pushing	<code>push</code> returns a number, and numbers have no <code>slice</code> — the first <code>console.log</code> threw.
<code>request.js</code> imports <code>apps/api/db.js</code>	The API injects a credential store (ADR 0003)	The original makes the import graph a cycle and forces every node test to need Postgres.
The canvas learns a run finished	The canvas watches it happen, over SSE (ADR 0004)	Nodes light up one at a time, exactly as n8n does.

Also new. A `getRawNodeParameter` on `ctx` so the Code node reads its program unrewritten; an error thrown by a node now has its node name attached by the engine; and `npm run migrate` replaces hand-typed `psql`, so nobody needs `psql` installed.

2 Everything we are building, on one page

Nine boxes. Every one of them is built in a phase you can point at, and nothing else exists.



Everything travels the same way: something starts a workflow, the engine walks the drawing box by box, each box does one job — often calling Google or a language model — and the run is written down.

3 The three ideas the whole thing rests on

A workflow is text

A list of boxes and a list of lines, in JSON. The canvas produces it, one database column stores it, the engine reads it. Because it is only text, the two halves of the project can be built at the same time by different people.

A node is one file with two halves

A *description* the screen turns into a settings form, and an `execute()` the engine calls. Neither half knows the other exists, which is why six people can add six nodes at once.

Everything travels as items

Always an array, even when it holds one thing. A node returns an array of *branches*. An empty branch goes quiet — and that single rule is how the false side of an IF stops everything after it.

a workflow, exactly as it is stored

```
{
  "nodes": [
    { "name": "Webhook", "type": "webhook",
      "parameters": { "path": "new-order", "method": "POST" },
      "position": { "x": 80, "y": 160 } },
    { "name": "Tidy up", "type": "set", "parameters": { "fields": [] } }
  ],
  "connections": { "Webhook": [ "Tidy up" ] }
}
```

`connections[from][branchIndex]` is the list of node **names** fed by that output. Branch 0 is the IF node's **true** side, and it equals React Flow's `sourceHandle` — one promise, agreed in week 6, holding the canvas and the engine together.

4 The layout everyone clones

the whole tree

```

ai-automation/
├── apps/
│   ├── api/          # Express, the routes, webhooks, schedules, OAuth  C1 C2
│   └── editor/        # React + React Flow + the parameter panel      B1 B2
├── packages/
│   ├── engine/        # the loop, items, expressions, errors, http helper  A1
│   └── nodes/
│       ├── core/      # IF, Set, Merge, Code, HTTP Request            A2
│       ├── google/    # Sheets, Drive, Docs, Gmail                    D1
│       └── ai/         # the LLM node                                  A1
├── benchmarks/        # k6 scripts, the n8n stack, raw results          D2
├── infra/              # Caddyfile, docker-compose.prod.yml, backup.sh    D2
├── scripts/            # doctor.js - the "why won't it start" answer machine C1
├── docs/               # this guide, the ADRs, the report as it is written  D2
├── e2e/                # the browser test                                B2
├── docker-compose.yml
├── schema.sql          # every table, in one file
├── .nvmrc              # 22 - so nvm picks the right Node with no arguing
└── .env.example

```

Five commands, ten minutes

a fresh clone

```

git clone <repo> && cd ai-automation
cp .env.example .env
npm install
docker compose up -d --wait
npm run seed && npm run dev

```

And if it does not start

`predev` runs the doctor first, every time, so a broken start can never happen silently. Each line names the problem *and the exact command that fixes it*.

npm run dev, on a teammate's first clone

```

ok Node 22 or newer      v22.22.2
ok Dependencies installed
FAIL .env exists          missing
-> run: cp .env.example .env
FAIL Postgres answers     ECONNREFUSED
-> run: docker compose up -d --wait

```

2 problem(s) - fix the arrows above.

5 Phase 1 · the engine

Three small files, and a workflow engine that refuses to lie to you when it breaks: it validates, it times out, it retries, and every failure names the node it came from.

One error type, so every failure names its node

`JSON.stringify(new Error('x'))` is `{}` — the message would be invisible in the database. Defining `toJson` is what lets us store the real failure and show it in the sidebar three days later.

`packages/engine/errors.js`

```
export class WorkflowError extends Error {
  constructor(message, { node = null, code = "s">'WORKFLOW_ERROR', cause = null } = {}) {
    super(message);
    this.name = "s">'WorkflowError';
    this.node = node;    "c"> // which node on the canvas
    this.code = code;    "c"> // machine-readable: the editor colours the box from this
    if (cause) this.cause = cause;
  }

  "c"> // JSON.stringify(new Error("s">'x')) is "{}" - the message would be invisible in the
  "c"> // database. Defining toJson is what lets us store the real failure and show it later.
  toJson() {
    return { name: this.name, code: this.code, node: this.node, message: this.message };
  }
}
```

One HTTP helper, used by every node forever

Write this once and no node author has to think about flaky networks again. The jitter matters: without it every retry in a batch fires at the same millisecond and knocks the far end over a second time.

```

export async function httpRequest(url, opts = {}) {
  const { retries = 3, timeoutMs = 30000, ...init } = opts;
  let lastError;

  for (let attempt = 0; attempt <= retries; attempt++) {
    try {
      const res = await fetch(url, { ...init, signal: AbortSignal.timeout(timeoutMs) });

      // 429 = "you are going too fast", 5xx = "my fault" - both are worth trying again
      if (res.status === 429 || res.status >= 500) {
        const after = Number(res.headers.get('retry-after')) * 1000;
        throw Object.assign(new Error(`HTTP ${res.status}`), {
          retryAfter: Number.isFinite(after) ? after : 0,
          status: res.status,
        });
      }

      // 4xx = "your fault" - retrying changes nothing, so stop immediately
      if (!res.ok) {
        const body = (await res.text()).slice(0, 300);
        throw Object.assign(new Error(`HTTP ${res.status}: ${body}`), { fatal: true, status: res.status });
      }

      if (res.status === 204) return {};
      const type = res.headers.get('content-type') || '';
      return type.includes('json') ? await res.json() : { data: await res.text() };
    } catch (err) {
      lastError = err;
      if (err.fatal || attempt === retries) break;
      // exponential backoff with jitter: 0.5s, 1s, 2s ... plus a random nudge so eight
      // parallel items do not all wake up and hammer the server at the same millisecond
      await sleep(err.retryAfter || Math.min(8000, 2 ** attempt * 500) + Math.random() * 250);
    }
  }
  throw lastError;
}

```

6 Phase 1 · the loop, and its guards

Refuse a bad workflow before any of it runs

A misspelled node name should be a clear sentence, not a crash halfway through a run that has already sent three emails.

packages/engine/engine.js

```
export function validateWorkflow(workflow, nodeTypes) {
  const problems = [];
  const nodes = Array.isArray(workflow?.nodes) ? workflow.nodes : [];
  if (nodes.length === 0) problems.push(class="s">'the workflow has no nodes');

  const names = new Set();
  for (const n of nodes) {
    if (!n?.name) problems.push(class="s">'a node has no name');
    else if (names.has(n.name)) problems.push(`two nodes are both called "${n.name}"`);
    else names.add(n.name);
    if (n?.name && !nodeTypes[n.type]) problems.push(`${n.name} has unknown type "${n.type}"`);
  }

  for (const [from, branches] of Object.entries(workflow?.connections ?? {})) {
    if (!names.has(from)) problems.push(`a connection starts at missing node "${from}"`);
    for (const targets of branches ?? []) {
      for (const to of targets ?? []) {
        if (!names.has(to)) problems.push(`${from} points at missing node "${to}"`);
      }
    }
  }

  const triggers = nodes.filter((n) => nodeTypes[n.type]?.description?.group === class="s">'trigger');
  if (triggers.length !== 1) {
    problems.push(`a workflow needs exactly one trigger - this one has ${triggers.length}`);
  }

  class="c"> // A loop has no valid running order, so it can never be executed. Saying so here
  class="c"> // costs nothing; discovering it at step 1000 costs a demo.
  if (problems.length === 0) {
    const cycle = findCycle(workflow);
    if (cycle) problems.push(`this workflow loops: ${cycle.join(class="s">' -> ')} `);
  }

  if (problems.length) {
    throw new WorkflowError(class="s">'This workflow cannot run:\n - ' + problems.join(class="s">'\\n - '),
      { code: class="s">'INVALID_WORKFLOW' });
  }
  return triggers[0];
}
```

Two guards the loop cannot do without

packages/engine/engine.js

```
function withTimeout(promise, ms, message) {
  let timer;
  const bell = new Promise( (_, reject) => {
    timer = setTimeout(() => reject(new WorkflowError(message, { code: class="s">'NODE_TIMEOUT' })), ms);
  });
  return Promise.race([promise, bell]).finally(() => clearTimeout(timer));
}
```



```

function normalise(branches, nodeName, maxItems) {
  if (!Array.isArray(branches)) {
    throw new WorkflowError(`"${nodeName}" must return an array of branches`,
      { node: nodeName, code: class="s">'BAD_NODE_OUTPUT' });
  }
  let total = 0;
  return branches.map((branch, i) => {
    if (!Array.isArray(branch)) {
      throw new WorkflowError(`branch ${i} of "${nodeName}" is not an array`,
        { node: nodeName, code: class="s">'BAD_NODE_OUTPUT' });
    }
    total += branch.length;
    if (total > maxItems) {
      throw new WorkflowError(`"${nodeName}" produced more than ${maxItems} items - refusing to go on`,
        { node: nodeName, code: class="s">'TOO_MANY_ITEMS' });
    }
    return branch.map((item) =>
      item && typeof item === class="s">'object' && class="s">'json' in item ? item : { json: item ?? {} });
  });
}

```

7 Phase 1 · the running order

The obvious loop keeps a ready list and pushes each target as soon as a branch produces items. It is three lines shorter, and it runs a node **once per incoming line** — which means Merge, a node we ship, runs twice with half its data each time.

packages/engine/engine.js

```
function runningOrder(workflow, triggerName) {
  const targetsOf = (name) => (workflow.connections?.[name] ?? []).flatMap((t) => t ?? []);

  class="c"> // only what the trigger can reach: a second, disconnected island must not run
  const reachable = new Set([triggerName]);
  const stack = [triggerName];
  while (stack.length) {
    for (const next of targetsOf(stack.pop())) {
      if (!reachable.has(next)) { reachable.add(next); stack.push(next); }
    }
  }

  const indegree = new Map([...reachable].map((n) => [n, 0]));
  for (const name of reachable) {
    for (const to of targetsOf(name)) {
      if (reachable.has(to)) indegree.set(to, indegree.get(to) + 1);
    }
  }

  class="c"> // seed in declaration order, so two runs of the same workflow choose the same order
  const queue = (workflow.nodes ?? []).map((n) => n.name).filter((n) => indegree.get(n) === 0);
  const order = [];
  while (queue.length) {
    const name = queue.shift();
    order.push(name);
    for (const to of targetsOf(name)) {
      if (!reachable.has(to)) continue;
      indegree.set(to, indegree.get(to) - 1);
      if (indegree.get(to) === 0) queue.push(to);
    }
  }
  return order;
}
```

The cost, stated plainly. A cycle has no topological order, so loops inside a workflow are impossible. n8n allows them; we do not. Iterating over items is the Code node's job instead. This is ADR 0002.

8 Phase 1 · what a node is handed

Frozen in week 3, and never changed since. Every node ever written depends on these names, which is what lets seven people write nodes against a written promise rather than against code that does not exist yet.

packages/engine/engine.js

```
const ctx = {
  getInputData: () => items,
  getNodeParameter(name, i = 0, fallback = undefined) {
    const raw = node.parameters?.[name] ?? fallback;
    if (raw === undefined) {
      throw new WorkflowError(`${nodeName}" needs its "${name}" setting filled in`,
        { node: nodeName, code: class="s">'MISSING_PARAMETER' });
    }
    return resolve(raw, items[i], results, i, items);      class="c"> // expressions, for free
  },
  class="c"> // The Code node needs its program exactly as typed - see packages/nodes/core/code.js.
  getRawNodeParameter(name, fallback = undefined) {
    return node.parameters?.[name] ?? fallback;
  },
  helpers: { httpRequest },
  logger: {
    info: (msg) => { log.push({ node: nodeName, level: class="s">'info', msg }); emit({ type:
class="s">'node-log', node: nodeName, msg }); },
    warn: (msg) => { log.push({ node: nodeName, level: class="s">'warn', msg }); emit({ type:
class="s">'node-log', node: nodeName, msg }); },
  },
  credentialId: node.credentials?.id ?? null, class="c"> // undefined settings become null, never crash
  credentials: options.credentials ?? {},
  workflow: { id: workflow.id, name: workflow.name },
  node: { name: nodeName, type: node.type },
};
```

NAME	WHAT IT IS
<code>getInputData()</code>	the items that arrived
<code>getNodeParameter(name, i, fallback)</code>	one setting, with expressions already resolved for item <code>i</code>
<code>getRawNodeParameter(name, fallback)</code>	the setting exactly as typed — the Code node needs its program unrewritten
<code>helpers.httpRequest(url, opts)</code>	the shared helper: timeout, retries, jitter
<code>logger.info(msg)</code>	one line in the run's receipt
<code>credentialId</code>	which credential the sidebar picked, or <code>null</code>

9 Phase 3 · expressions, resolved per item

Any setting may contain `{{ ... }}`, and the engine fills it in for every item separately, just before the node runs. Two items in, two different requests out. Node authors never see braces.

packages/engine/expressions.js

```
const SHADOWED = [
  class="s">'process', class="s">'require', class="s">'module', class="s">'exports', class="s">'globalThis',
  class="s">'global',
  class="s">'fetch', class="s">'Function', class="s">'setTimeout', class="s">'setInterval',
  class="s">'XMLHttpRequest', class="s">'WebSocket', class="s">'constructor',
];
```

```
const FORBIDDEN = /\b(?:eval|arguments)\b/;
```

packages/engine/expressions.js

```
export function resolve(value, item, results = {}, itemIndex = 0, items = []) {
  class="c"> // Expressions may be nested inside objects and arrays - a JSON parameter such as
  class="c"> // { "name": "{{ $json.name }}" } has to work exactly like a plain text box does.
  if (Array.isArray(value)) return value.map((v) => resolve(v, item, results, itemIndex, items));
  if (value && typeof value === class="s">'object') {
    const out = {};
    for (const [k, v] of Object.entries(value)) out[k] = resolve(v, item, results, itemIndex, items);
    return out;
  }
  if (typeof value !== class="s">'string' || !value.includes(class="s">'{{')) return value;

  const evaluate = (expr) => {
    const $json = item?.json ?? {};
    const $node = new Proxy({}, {
      get: (_, name) => {
        const branches = results[name];
        if (!branches) {
          throw new WorkflowError(`$node["${String(name)}"] has not run yet`, { code:
            class="s">'BAD_EXPRESSION' });
        }
        return { json: branches[0]?.[0]?.json ?? {}, all: branches.flat() };
      },
    });
  });
  try {
    return compile(expr.trim())($json, $node, new Date(), itemIndex, items);
  } catch (e) {
    if (e instanceof WorkflowError) throw e;
    throw new WorkflowError(`{{${expr.trim()}} -> ${e.message}`, { code: class="s">'BAD_EXPRESSION' });
  }
};

const whole = value.match(/^s*\{\{([s\S]+)\}\}s*$/); class="c"> // the field is ONE expression
if (whole) return evaluate(whole[1]); class="c"> // ...so keep the real type
return value.replace(/\{\{([s\S]+?)\}\}/g, (_, e) => {
  const out = evaluate(e);
  return out === null || out === undefined ? class="s">'': typeof out === class="s">'object' ?
JSON.stringify(out) : String(out);
});
}
```

Say this in the report, in these words. Shadowing is a speed bump, not a sandbox. Anybody who can edit a workflow can already reach the network through the HTTP node. The real isolation story is the Code node — and it has limits too.

10 Phase 2 · the database, and the five status codes

schema.sql

```
CREATE TABLE IF NOT EXISTS workflows (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL CHECK (length(trim(name)) BETWEEN 1 AND 120),
  active       boolean NOT NULL DEFAULT false,
  nodes        jsonb NOT NULL DEFAULT class="s">'[]' CHECK (jsonb_typeof(nodes) = class="s">'array'),
  connections  jsonb NOT NULL DEFAULT class="s">'{}' CHECK (jsonb_typeof(connections) = class="s">'object'),
  version      integer NOT NULL DEFAULT 1,          -- bumped on every save; stops silent overwrites
  created_at   timestampz NOT NULL DEFAULT now(),
  updated_at   timestampz NOT NULL DEFAULT now()
);

-- --- every past run -----
CREATE TABLE IF NOT EXISTS executions (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workflow_id uuid NOT NULL REFERENCES workflows(id) ON DELETE CASCADE,
  mode        text NOT NULL DEFAULT class="s">'manual'
              CHECK (mode IN (class="s">'manual', class="s">'webhook', class="s">'schedule')),
  status      text NOT NULL CHECK (status IN
(class="s">'running', class="s">'success', class="s">'error', class="s">'cancelled')),
  data        jsonb,                      -- what each node produced
  log         jsonb NOT NULL DEFAULT class="s">'[]',      -- the receipt from the engine
  error       jsonb,
  started_at  timestampz NOT NULL DEFAULT now(),
  finished_at timestampz
);

-- the executions list is the most-hit query in the whole product: index it now
CREATE INDEX IF NOT EXISTS executions_by_workflow
  ON executions (workflow_id, started_at DESC);
CREATE INDEX IF NOT EXISTS executions_running
  ON executions (status) WHERE status = class="s">'running';

-- keep updated_at honest without every route remembering to set it
CREATE OR REPLACE FUNCTION touch_updated_at() RETURNS trigger AS $$
BEGIN NEW.updated_at = now(); NEW.version = OLD.version + 1; RETURN NEW; END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS workflows_touch ON workflows;
CREATE TRIGGER workflows_touch BEFORE UPDATE ON workflows
  FOR EACH ROW EXECUTE FUNCTION touch_updated_at();
```

CHECK constraint. A rule the database itself enforces. Any code path, any teammate, any hand-typed psql command: a row that breaks the rule is simply rejected. It is the cheapest bug prevention in the project.

CODE	WHEN	WHAT THE CALLER IS TOLD
400	no name, or the id is not a UUID	name is required
404	no workflow with that id	No workflow with that id
409	somebody else saved first	Somebody else saved this workflow - reload before saving
409	two workflows claim one webhook address	Another workflow already listens on POST /webhook/x

CODE	WHEN	WHAT THE CALLER IS TOLD
422	the engine refused the workflow	<pre>{ error: { code: 'INVALID_WORKFLOW', node, message } }</pre>

Those five numbers are the deliverable, not the 201. Anyone can write a route that works; a marker looks at what happens when it does not. Each of them has a test in `apps/api/server.test.js`.

11 Phase 2 · saving, without losing anybody's work

The editor sends the `version` it loaded. If somebody else saved in between, the UPDATE matches nothing and we say so, instead of eating their changes.

apps/api/server.js

```
app.patch(class="s">' /rest/workflows/:id', route(async (req, res) => {
  const id = requireId(req);
  const wf = validateWorkflowBody(req.body, { partial: true });

  class="c"> // optimistic locking: the editor sends the version it loaded. If somebody else saved
  class="c"> // in between, the UPDATE matches nothing and we say so instead of eating their work.
  const { rows } = await db.query(
    `UPDATE workflows SET name          = COALESCE($2, name),
                           nodes        = COALESCE($3, nodes),
                           connections   = COALESCE($4, connections)
    WHERE id = $1 AND ($5::int IS NULL OR version = $5) RETURNING *`,
    [id, wf.name ?? null,
      wf.nodes ? JSON.stringify(wf.nodes) : null,
      wf.connections ? JSON.stringify(wf.connections) : null,
      req.body.version ?? null]);

  if (!rows[0]) {
    const { rowCount } = await db.query(class="s">'SELECT 1 FROM workflows WHERE id = $1', [id]);
    if (!rowCount) throw new HttpError(404, class="s">'No workflow with that id');
    throw new HttpError(409, class="s">'Somebody else saved this workflow - reload before saving');
  }
  res.json(rows[0]);
}));
```


12 Phase 2 · our format $\rightleftharpoons$ React Flow's

These two functions are where every canvas bug would live for the rest of the semester. They are written defensively, and tested in both directions.

apps/editor/src/convert.ts

```
export function toReactFlow(wf: Workflow): { nodes: RfNode[]; edges: Edge[] } {
  const nodes: RfNode[] = (wf.nodes ?? []).map((n, i) => ({
    id: n.name,
    type: class="s">'workflowNode',
    position: n.position ?? fallbackPosition(i),
    data: { node: n },
    dragHandle: undefined,
  }));

  const known = new Set(nodes.map((n) => n.id));

  const edges: Edge[] = Object.entries(wf.connections ?? {}).flatMap(([from, branches]) =>
    (branches ?? []).flatMap((targets, branchIndex) =>
      (targets ?? [])
        .class="c"> // a connection to a deleted node would crash React Flow - drop it quietly
          .filter((to) => known.has(to) && known.has(from))
          .map((to) => ({
            id: `${from}:${branchIndex}->${to}`,
            source: from,
            target: to,
            sourceHandle: String(branchIndex),
            targetHandle: class="s">'in',
            type: class="s">'workflowEdge',
          })))));

  return { nodes, edges };
}
```

apps/editor/src/convert.ts

```
export function fromReactFlow(nodes: RfNode[], edges: Edge[]): Pick<Workflow, class="s">'nodes' |
class="s">'connections'> {
  const connections: Record<string, string[][]> = {};

  for (const e of edges) {
    const branch = Number.isFinite(Number(e.sourceHandle)) ? Number(e.sourceHandle) : 0;
    connections[e.source] ??= [];
    for (let b = 0; b <= branch; b++) connections[e.source][b] ??= []; class="c"> // no holes in the array
    if (!connections[e.source][branch].includes(e.target)) { class="c"> // no duplicate edges
      connections[e.source][branch].push(e.target);
    }
  }

  return {
    nodes: nodes.map((n) => ({
      ...((n.data as { node: WorkflowNode }).node),
      name: n.id,
      position: { x: Math.round(n.position.x), y: Math.round(n.position.y) },
    })),
    connections,
  };
}
```

The bug this catches. A connection to a node somebody deleted would crash React Flow. It is dropped quietly instead, and `isLossless()` proves a workflow survives the round trip unchanged.

13 Phase 3 · a form nobody wrote

One component turns any node description into a settings panel, validates it, and previews every expression in it. Adding a field to a node means adding a line to that node's `properties` array — the editor never changes.

apps/editor/src/components/ParameterPanel.tsx

```
export function isVisible(p: Property, values: Record<string, unknown>): boolean {
  const { show, hide } = p.displayOptions ?? {};
  const matches = (rules: Record<string, unknown[]>) =>
    Object.entries(rules).every(([field, allowed]) => allowed.includes(values[field]));
  if (show && !matches(show)) return false;
  if (hide && matches(hide)) return false;
  return true;
}
```

apps/editor/src/components/ParameterPanel.tsx

```
export function validateParameter(p: Property, value: unknown): string | null {
  const isExpression = typeof value === 'class="s">'string' && value.includes('class="s">{');
  if (isExpression) return null;
  if (p.required && (value === 'class="s">' || value === null || value === undefined)) {
    return `${p.displayName} is required`;
  }
  if (!p.required && (value === 'class="s">' || value === null || value === undefined)) return null;

  const v = p.validate ?? {};
  if (v.pattern && !new RegExp(v.pattern).test(String(value ?? 'class="s">'))) {
    return v.message ?? `${p.displayName} is not in the right format`;
  }
  if (p.type === 'class="s">'number') {
    if (!Number.isFinite(Number(value))) return `${p.displayName} must be a number`;
    if (v.min !== null && Number(value) < v.min) return `${p.displayName} must be at least ${v.min}`;
    if (v.max !== null && Number(value) > v.max) return `${p.displayName} must be at most ${v.max}`;
  }
  if (p.type === 'class="s">'json') {
    try {
      JSON.parse(typeof value === 'class="s">'string' ? (value || 'class="s">{') : JSON.stringify(value));
    } catch {
      return `${p.displayName} is not valid JSON`;
    }
  }
  return null;
}
```

What it produces

Change Method from GET to POST and a Body field appears — because the node said `displayOptions: { show: { method: ['POST', 'PUT', 'PATCH'] } }`, and for no other reason.

The preview

Type `{{ $json.city }}` and the panel shows what it will become, using **the same resolver the engine uses**. The preview cannot disagree with what will actually happen.

14 A node, whole — the shape all thirteen share

packages/nodes/core/set.js

```
export const setNode = {
  description: {
    name: class="s">'set',
    displayName: class="s">'Set',
    group: class="s">'transform',
    icon: class="s">'pencil',
    colour: class="s">'#0ea5e9',
    description: class="s">'Add, rename or overwrite fields on every item',
    inputs: [class="s">'main'],
    outputs: [class="s">'main'],
    properties: [
      { displayName: class="s">'Keep only set fields', name: class="s">'keepOnlySet', type:
class="s">'boolean', default: false,
        hint: class="s">'Drop everything that arrived and emit only the fields below' },
      { displayName: class="s">'Fields', name: class="s">'fields', type: class="s">'fields', default: [],
        hint: class="s">'Values may use expressions, e.g. {{ $json.first }} {{ $json.last }}' },
    ],
  },
  async execute(ctx) {
    const items = ctx.getInputData();
    const out = [];

    for (let i = 0; i < items.length; i++) {
      const keepOnlySet = ctx.getNodeParameter(class="s">'keepOnlySet', i, false);
      const fields = ctx.getNodeParameter(class="s">'fields', i, []) ?? [];
      const json = keepOnlySet ? {} : structuredClone(items[i].json ?? {});

      for (const field of fields) {
        if (!field?.name) continue;
        setPath(json, field.name, coerce(field.value, field.type));
      }
      out.push({ json, pairedItem: i });
    }
    return [out];
  },
};
```

Adding a node is four steps. Write the file · add two lines to `packages/nodes/index.js` · write its test beside it · restart and refresh. The palette entry and the whole settings panel are already there. Nobody touches the editor.

15 Phase 4 · webhooks — the one address strangers can reach

Everything else in this project sits behind our own editor. This does not. Every line of it treats the request as hostile, because it is.

```

export function mountWebhooks(app) {
  app.all(class="s">' /webhook/:path', hookLimit, rawJson, rawForm, async (req, res) => {
    try {
      const isTest = req.query.test === class="s">'1';
      const { rows: [hook] } = await db.query(
        class="s">'SELECT * FROM webhooks WHERE path = $1 AND method = $2 AND is_test = $3',
        [req.params.path, req.method, isTest]);

      if (!hook) return res.status(404).json({ message: class="s">'No workflow is listening here' });

      class="c"> // optional shared secret, compared in constant time so timing cannot leak it
      if (hook.secret) {
        const expected = crypto.createHmac(class="s">'sha256', decrypt(hook.secret))
          .update(req.rawBody ?? Buffer.alloc(0)).digest(class="s">'hex');
        const given = String(req.get(class="s">'x-signature') ?? class="s">'');
        const ok = given.length === expected.length
          && crypto.timingSafeEqual(Buffer.from(given), Buffer.from(expected));
        if (!ok) return res.status(401).json({ message: class="s">'Bad or missing signature' });
      }

      class="c"> // deduplicate retries from the sender: Stripe, GitHub and Shopify all retry
      class="c"> // a webhook they think failed, and the work must not happen twice
      const key = req.get(class="s">'idempotency-key') || req.get(class="s">'x-github-delivery');
      if (key) {
        const { rows } = await db.query(
          class="s">'SELECT execution_id FROM webhook_deliveries WHERE idempotency_key = $1', [key]);
        if (rows[0]) {
          return res.status(200).json({ message: class="s">'Already handled', executionId:
            rows[0].execution_id });
        }
      }

      const triggerItems = [{
        json: {
          headers: req.headers,
          query: req.query,
          body: req.body ?? {},
          method: req.method,
          receivedAt: new Date().toISOString(),
        },
      }];

      if (hook.response_mode === class="s">'immediately') {
        res.status(202).json({ message: class="s">'Workflow was started' }); class="c"> // answer in ~5 ms
        runById(hook.workflow_id, triggerItems, class="s">'webhook')
          .then((r) => key && remember(key, r.executionId))
          .catch((err) => console.error(class="s">'[webhook]', err.message)); class="c"> // never crash the
        process
        return;
      }

      const result = await runById(hook.workflow_id, triggerItems, class="s">'webhook');
      if (key) await remember(key, result.executionId);
      return res.json(lastNodeOutput(result.results));
    }
  });
}

```

Idempotency. “Doing it twice has the same effect as doing it once.” Stripe, GitHub and Shopify all retry a webhook they think failed. Without the deliveries table a retried order becomes two spreadsheet rows and two emails.

16 Phase 4 · the clock, and the three guards it needs

apps/api/schedules.js

```
export function activate(workflow, schedule) {
  deactivate(workflow.id);
  validateSchedule(schedule);

  const job = new Cron(
    schedule.cron,
    {
      timezone: schedule.timezone,
      protect: true, class="c"> // 1 - if the last run is still going, SKIP this tick, never pile up
      catch: (err) => console.error(`[schedule ${workflow.name}]`, err.message), class="c"> // 2 - never
    },
    () => runById(workflow.id, [{
      json: {
        timestamp: new Date().toISOString(),
        scheduledFor: job.currentRun()?.toISOString() ?? null,
      },
    }], class="s">'schedule'),
  );

  jobs.set(workflow.id, job);
  console.log(`[schedule] ${workflow.name} -> next ${job.nextRun()?.toISOString()}`);
  return job.nextRun();
}
```

protect: true is the most valuable option on this page. A workflow set to every minute that takes ninety seconds will, without it, start a second copy before the first finishes, then a third, until the machine dies. With it, the tick is skipped and logged.

apps/api/schedules.js

```
export async function restoreAll() {
  const { rows } = await db.query(
    class="s">'SELECT w.*, s.cron, s.timezone FROM workflows w JOIN schedules s ON s.workflow_id = w.id '
    + class="s">'WHERE w.active = true');
  let restored = 0;
  for (const row of rows) {
    try {
      activate(row, row);
      restored += 1;
    } catch (e) {
      console.error(`[schedule] ${row.name} is broken: ${e.message}`);
    }
  }
  console.log(`restored ${restored} schedule(s)`);
  return restored;
}
```

17 Phase 5 · keys you can trust, in twenty-five lines

apps/api/crypto.js

```
export function encrypt(text) {
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv(class="s">'aes-256-gcm', key, iv);
  const data = Buffer.concat([cipher.update(String(text), class="s">'utf8'), cipher.final()]);
  return {
    v: 1,
    iv: iv.toString(class="s">'base64'),
    tag: cipher.getAuthTag().toString(class="s">'base64'),
    data: data.toString(class="s">'base64'),
  };
}
```

apps/api/crypto.js

```
export function decrypt(envelope) {
  if (!envelope || envelope.v !== 1) throw new Error(class="s">'Unknown credential format');
  const decipher = crypto.createDecipheriv(class="s">'aes-256-gcm', key, Buffer.from(envelope.iv,
class="s">'base64'));
  decipher.setAuthTag(Buffer.from(envelope.tag, class="s">'base64'));
  try {
    return Buffer.concat([
      decipher.update(Buffer.from(envelope.data, class="s">'base64')),
      decipher.final(),
    ]).toString(class="s">'utf8');
  } catch {
    throw new Error(class="s">'Credential could not be decrypted - has ENCRYPTION_KEY changed?');
  }
}
```

GCM is the authenticated mode of AES. It does not only hide the token, it detects tampering: change one byte in the database and `final()` throws instead of quietly returning rubbish. That is what the `tag` field is for.

18 Phase 5 · one helper, then four Google nodes

D1 writes this first and freezes it. After that each node is a settings description plus one request, and four people can work in parallel without talking to each other.

packages/nodes/google/request.js

```
export async function googleRequest(credentialId, url, options = {}) {
  class="c"> // catch the empty sidebar field here, with a clear message - not four calls deep,
  class="c"> // reported as "credential has been deleted" for something that was never set
  if (!credentialId) throw new Error(class="s">'This node has no credential selected - pick one in the sidebar');

  const { retries = 4, timeoutMs = 30_000, ...init } = options;
  let token = await getAccessToken(credentialId);

  for (let attempt = 0; attempt <= retries; attempt++) {
    const res = await fetch(url, {
      ...init,
      signal: AbortSignal.timeout(timeoutMs),
      headers: { class="s">'content-type': class="s">'application/json', ...init.headers, Authorization: `Bearer ${token}` },
    });

    if (res.ok) return res.status === 204 ? {} : res.json();

    class="c"> // 401 - the hour is up. Refresh once and try the same request again.
    if (res.status === 401 && attempt === 0) {
      token = await refresh(credentialId);
      continue;
    }

    const body = await res.text();

    class="c"> // 403 "insufficient permissions" is a SCOPE problem, and no amount of retrying
    class="c"> // fixes it. Say the real cause instead of "Google 403".
    if (res.status === 403 && /insufficient|scope/i.test(body)) {
      throw new Error(class="s">'This credential was not granted the scope this node needs. '
        + class="s">'Reconnect it and tick the missing permission.');
```

Single-flight. When ten items discover the token expired at the same millisecond, ten refresh requests go to Google, nine of them invalidate each other, and the node fails for no visible reason. One shared promise in a Map fixes it in three lines and is worth a paragraph in the report.

19 Phase 5 · Sheets — fifty items, one request

The obvious version sends one request per item. Fifty items is fifty round trips and a rate limit. Collect first, send once — and because it is measurable, the report measures it.

packages/nodes/google/sheets.js

```
class="c"> // read the header row once, so column order follows the sheet, not the object
const header = await googleRequest(ctx.credentialId, valuesUrl(`${sheet}!1:1`));
const columns = header.values?.[0] ?? null;

const values = items.map((_, i) => {
  const row = ctx.getNodeParameter(class="s">'columns', i, {}); class="c"> // { name, email, ... }
  const parsed = typeof row === class="s">'string' ? JSON.parse(row || class="s">'{}') : row;
  return columns ? columns.map((c) => stringify(parsed[c])) : Object.values(parsed).map(stringify);
});

class="c"> // Sheets caps a write at 10 MB / ~10 000 rows; chunk so a big run cannot fail as a whole
const responses = [];
for (let start = 0; start < values.length; start += 500) {
  responses.push(await googleRequest(
    ctx.credentialId,
    valuesUrl(`${sheet}!A:Z`, class="s">':append', class="s">'?
valueInputOption=USER_ENTERED&insertDataOption=INSERT_ROWS'),
    { method: class="s">'POST', body: JSON.stringify({ values: values.slice(start, start + 500) }) },
  ));
}

ctx.logger.info(`appended ${values.length} row(s) in ${responses.length} request(s)`);
return [items.map((item, i) => ({
  json: { ...item.json, updatedRange: responses[0]?.updates?.updatedRange ?? null },
  pairedItem: i,
```

What it looks like from the user's side. Three items in, one API call out, and three rows in the spreadsheet with identical timestamps — because it was a single append, not three. The same code handles five hundred.

20 Phase 5 · the Code node, and an honest admission

Two limits, not one. A timeout stops slow code; only a memory limit stops `const a = []; while (true) a.push('x')` from taking the server with it.

```

export function runUserCode(code, items, { timeoutMs = 5000, memoryMb = 64 } = {}) {
  return new Promise((resolve, reject) => {
    if (typeof code !== class="s">'string' || code.trim() === class="s">'') {
      return reject(new Error(class="s">'The Code node is empty'));
    }

    const worker = new Worker(WORKER, {
      eval: true,
      workerData: { code, items, timeoutMs },
      resourceLimits: { maxOldGenerationSizeMb: memoryMb, maxYoungGenerationSizeMb: 16 },
      stdout: true, stderr: true, class="c"> // user code cannot spam our server log
    });

    let settled = false;
    const finish = (fn, value) => {
      if (settled) return;
      settled = true;
      clearTimeout(timer);
      worker.terminate();
      fn(value);
    };

    class="c"> // the vm timeout can be beaten by a tight async loop, so the thread gets a hard deadline too
    const timer = setTimeout(
      () => finish(reject, new Error(`Code node timed out after ${timeoutMs} ms`)),
      timeoutMs + 500,
    );

    worker.on(class="s">'message', (m) => {
      if (!m.ok) {
        class="c"> // V8's own wording for the vm timeout is "Script execution timed out",
        class="c"> // which does not say which node or how long it was given.
        return finish(reject, /timed out/i.test(m.error)
          ? new Error(`Code node timed out after ${timeoutMs} ms`)
          : new Error(`Code node: ${m.error}`));
      }
      if (!Array.isArray(m.result)) {
        return finish(reject, new Error(class="s">'Code must return an array of items, e.g. return items;'));
      }
      finish(resolve, {
        items: m.result.map((x) => (x && typeof x === class="s">'object' && class="s">'json' in x ? x : {
json: x ?? {} })),
        logs: m.logs ?? [],
      });
    });

    worker.on(class="s">'error', (e) => finish(reject,
      /heap out of memory|Worker terminated|Array buffer allocation failed/i.test(e.message)
        ? new Error(`Code node used more than ${memoryMb} MB and was stopped`)
        : e));

    worker.on(class="s">'exit', (exitCode) => {
      if (!settled && exitCode !== 0) {
        finish(reject, exitCode === 1
          ? new Error(`Code node used more than ${memoryMb} MB and was stopped`)
          : new Error(`Code node exited unexpectedly (${exitCode})`));
      }
    });
  });
}

```

The wall we built, and the wall we did not. A determined attacker can still climb out of a `vm` context. A production platform uses `isolated-vm`, or a container per execution. Being able to explain the limits of our own sandbox is worth more marks than quietly using a stronger one would have been.

21 Phase 5 · the AI node

packages/nodes/ai/anthropic.js

```

async function ask(ctx, i) {
  const apiKey = ctx.credentials?.anthropicApi?.apiKey ?? process.env.ANTHROPIC_API_KEY;
  const baseUrl = ctx.credentials?.anthropicApi?.baseUrl
    ?? process.env.ANTHROPIC_BASE_URL ?? `class="s">'https://api.anthropic.com';

  if (!apiKey || apiKey === `class="s">'replace-me') {
    throw new Error(`class="s">'No API key for the AI node - set ANTHROPIC_API_KEY in .env, or connect a
    credential');
  }

  const prompt = String(ctx.getNodeParameter(`class="s">'userPrompt', i));
  if (prompt.length > 20_000) throw new Error(`class="s">'Prompt is too long - trim the data you send in');

  const body = {
    model: ctx.getNodeParameter(`class="s">'model', i, `class="s">'claude-sonnet-5'),
    max_tokens: Math.min(Math.max(Number(ctx.getNodeParameter(`class="s">'maxTokens', i, 512)), 1), 4096),
    messages: [{ role: `class="s">'user', content: prompt }],
  };

  const system = ctx.getNodeParameter(`class="s">'systemPrompt', i, `class="s">'');
  if (system) body.system = system;

  for (let attempt = 0; attempt < 4; attempt++) {
    const res = await fetch(`${baseUrl}/v1/messages`, {
      method: `class="s">'POST',
      signal: AbortSignal.timeout(60_000),
      headers: {
        `class="s">'x-api-key': apiKey,
        `class="s">'anthropic-version': `class="s">'2023-06-01',
        `class="s">'content-type': `class="s">'application/json',
      },
      body: JSON.stringify(body),
    });

    `class="c"> // 429 too fast, 529 overloaded, 5xx their fault - all worth waiting out
    if (res.status === 429 || res.status === 529 || res.status >= 500) {
      const wait = Number(res.headers.get(`class="s">'retry-after')) * 1000 || 2 ** attempt * 1000;
      ctx.logger.info(`model busy (${res.status}) - waiting ${wait} ms`);
      await new Promise((r) => setTimeout(r, wait));
      continue;
    }

    const data = await res.json();
    if (!res.ok) throw new Error(`AI ${res.status}: ${data.error?.message ?? `class="s">'unknown error'}`);
    if (data.stop_reason === `class="s">'max_tokens') ctx.logger.warn(`class="s">'answer was cut off - raise
    Max tokens');

    const text = (data.content ?? []).filter((c) => c.type === `class="s">'text').map((c) =>
    c.text).join(`class="s">'');
    ctx.logger.info(`in ${data.usage?.input_tokens ?? `class="s">'?'`} / out ${data.usage?.output_tokens ??
    `class="s">'?'`} tokens`);
    return { text, usage: data.usage, model: data.model, stopReason: data.stop_reason };
  }

  throw new Error(`class="s">'The model stayed busy after four attempts');
}

```

```

export function parseJsonAnswer(text, expectedKeys = []) {
  const block = text.match(/``(?:json)?s*([\s\S]*?)``/)?.[1] ?? text;
  const slice = block.slice(block.indexOf('s">{''), block.lastIndexOf('s">}') + 1);

  let parsed;
  try {
    parsed = JSON.parse(slice);
  } catch {
    throw new Error(`The model did not return JSON. It said: ${text.slice(0, 120)}...`);
  }

  const missing = expectedKeys.filter((k) => !(k in parsed));
  if (missing.length) throw new Error(`The answer is missing: ${missing.join('s">', ' ')}`);
  return parsed;
}

```

The seam to agree. Everything downstream reads `$json.urgency`. Change the prompt to emit `priority` and three workflows break silently — so the key lives in the node's default prompt.

22 Four levels of testing, and what each one catches

LEVEL	WHAT IT CATCHES	HOW LONG	WHERE
1 · engine	order, branch pruning, every guard	milliseconds	<code>packages/engine/*.test.js</code>
2 · nodes	the right request, built from the settings — against a fake server	seconds	<code>packages/nodes/**/*.test.js</code>
2b · API	the five status codes, webhooks, idempotency, schedules	a second	<code>apps/api/server.test.js</code>
3 · browser	drag, connect, run, reload	seconds	<code>e2e/build-and-run.spec.ts</code>
4 · by hand	a real email; a real spreadsheet row	before every demo	<code>docs/demo-checklist.md</code>

npm test – no internet, no Google credentials

```

✓ packages/engine/engine.test.js      (26 tests)
✓ packages/engine/http.test.js        (11 tests)
✓ packages/nodes/core/flow.test.js    (41 tests)
✓ packages/nodes/core/code.test.js    (12 tests)
✓ packages/nodes/google/sheets.test.js (10 tests)
✓ packages/nodes/google/gmail.test.js (12 tests)
✓ apps/api/server.test.js              (20 tests)
✓ apps/editor/src/convert.test.ts     (10 tests)

```

```

Test Files  8 passed (8)
   Tests    142 passed (142)
   Duration  6.5 s

```

coverage – branch coverage is the honest number

File	% Stmts	% Branch	% Funcs
engine.js	96.51	79.64	82.35
http.js	100.00	90.47	100.00
expressions	88.17	80.55	80.00
if.js	100.00	90.38	100.00
convert.ts	97.10	72.50	100.00

The exercise worth doing before the demo. Delete one guard from `engine.js` — say the step limit — and confirm a test goes red. A suite that stays green when you break the code is worse than no suite.

23 Level 3 — one browser test, and only one

It walks every seam in one go: canvas → server → engine → node → back to the canvas. Browser tests are slow and break for boring reasons, so there is exactly one that matters.

e2e/build-and-run.spec.ts

```
test(class=>'draw two nodes, connect them, run, and see the result', async ({ page, request }) => {
  const id = await newWorkflow(request, `e2e ${Date.now()}`);
  await page.goto(`/#/workflow/${id}`);

  await expect(page.getByTestId(class=>'canvas')).toBeVisible();
  await expect(page.getByTestId(class=>'palette')).toBeVisible();

  class=>'c' // 1 - build the workflow by clicking, exactly as a person would
  await page.getByTestId(class=>'palette-manualTrigger').click();
  await expect(page.getByTestId(class=>'node-Manual Trigger')).toBeVisible();

  await page.getByTestId(class=>'palette-set').click();
  await expect(page.getByTestId(class=>'node-Set')).toBeVisible();

  class=>'c' // 2 - drag an arrow from the trigger class=>'s' output dot onto the Set node's input dot
  await connect(page, class=>'s'>'handle-Manual Trigger-0', class=>'s'>'target-Set');
  await expect(page.locator(class=>'s'>'.react-flow__edge')).toHaveCount(1);

  class=>'c' // 3 - the settings panel drew itself from the node's own description
  await page.getByTestId(class=>'node-Set').click();
  await expect(page.getByTestId(class=>'parameter-panel')).toContainText(class=>'s'>'Keep only set fields');
  await expect(page.getByTestId(class=>'parameter-panel')).toContainText(class=>'s'>'Fields');

  await page.getByTestId(class=>'s'>'add-field').click();
  await page.getByTestId(class=>'s'>'field-name-0').fill(class=>'s'>'greeting');
  await page.getByTestId(class=>'s'>'field-value-0').fill(class=>'s'>'hello from the browser test');
  await page.keyboard.press(class=>'s'>'Escape');      class=>'c' // close the panel, back to the canvas

  class=>'c' // 4 - save, and confirm the server accepted it
  await page.getByTestId(class=>'s'>'save').click();
  await expect(page.getByTestId(class=>'s'>'save-status')).toContainText(class=>'s'>'Saved');

  const saved = await (await request.get(`${API}/rest/workflows/${id}`)).json();
  expect(saved.connections).toEqual({ class=>'s'>'Manual Trigger': [[class=>'s'>'Set']] });

  class=>'c' // 5 - run it, and watch the boxes report back
  await page.getByTestId(class=>'s'>'run').click();
  await expect(page.getByTestId(class=>'s'>'node-Set')).toHaveAttribute(class=>'s'>'data-state',
class=>'s'>'success');
  await expect(page.getByTestId(class=>'s'>'node-Manual Trigger')).toHaveAttribute(class=>'s'>'data-state',
class=>'s'>'success');

  await page.getByTestId(class=>'s'>'node-Set').click();
  await expect(page.getByTestId(class=>'s'>'output-Set')).toContainText(class=>'s'>'hello from the browser test');

  class=>'c' // 6 - it really persisted. This is the bug the test exists to catch.
  await page.reload();
  await expect(page.getByTestId(class=>'s'>'node-Set')).toBeVisible();
  await expect(page.getByTestId(class=>'s'>'node-Manual Trigger')).toBeVisible();
  expect(await page.locator(class=>'s'>'.react-flow__edge').count()).toBe(1);
});
```

The boring reason it used to be flaky. Adding a node slides the canvas to show it. Measuring a handle mid-animation gives coordinates that are stale by the time the mouse arrives, so `canvasIsStill()` waits for the view to settle first.

24 Phase 6 · online, and measured against n8n

Three containers and a certificate

Caddy obtains and renews the Let's Encrypt certificate by itself. There is no step 2. Postgres has **no** published port: it is reachable only from the other containers.

infra/Caddyfile

```
your-domain.com {
  encode gzip

  header {
    Strict-Transport-Security "max-age=31536000"
    X-Content-Type-Options    "nosniff"
    Referrer-Policy          "strict-origin-when-
cross-origin"
    -Server
  }

  reverse_proxy api:5678
}
```

Measured honestly

The same three workflows, the same k6 script, the same CPU and memory limits, the pinned n8n version, three runs each — and every raw JSON file committed.

benchmarks/k6/w1-webhook.js

```
export const options = {
  scenarios: {
    warmup: { executor: 'constant-vus',
vus: 2, duration: '20s', tags: { phase:
'warmup' } },
    load: {
      executor: 'constant-vus', vus: 10,
duration: '60s', startTime:
'25s',
      gracefulStop: '10s', tags: { phase:
'load' },
    },
  },
  'c'> // the run FAILS if these are missed,
so a bad result cannot quietly be reported as good
thresholds: {
    's'>'http_req_duration{phase:load}':
[ 'p(95)<1000', 's'>'p(99)<2500' ],
    workflow_failed: [ 'rate<0.01' ],
    http_req_failed: [ 'rate<0.01' ],
  },
  summaryTrendStats: [ 'avg',
's'>'min', 's'>'med',
's'>'p(95)', 's'>'p(99)',
's'>'max' ],
};
```

The sentence that earns the marks. n8n does far more per run than we do — queue mode, two workers, binary data, a node library in the hundreds. Being faster mostly means we do less. An honest number that loses to n8n is worth more than a flattering one nobody can reproduce.

25 Every seam, and who owns it

Each arrow is a promise one pair makes to another. If a change would add a ninth arrow, that is a ten-minute group conversation — not a pull request somebody discovers on Thursday.

SEAM	SHAPE	OWNED BY
ctx	getInputData · getNodeParameter · getRawNodeParameter · helpers · logger · credentialId	A1 freezes it, week 3
node output	[[item, ...], [item, ...]] — one array per branch	A1 · A2
branch index	branch 0 is IF's true , and equals sourceHandle	A2 · B1 · B2
workflow JSON	{ id, name, nodes[], connections{}, version }	everyone
version	the canvas sends what it loaded; the server answers 409 if stale	B1 · C1
trigger item	{ headers, query, body, method, receivedAt }	C2
credential	node.credentials.id on the workflow JSON	B1 · C1 · D1
live events	node-started · node-finished · node-error · node-skipped	A1 · B2

AREA	FOLDER	WHO
The engine	packages/engine/	A1
Core nodes	packages/nodes/core/	A2
The canvas	apps/editor/ — canvas and executions list	B1
The sidebar	apps/editor/ — parameter panel and expressions	B2
Routes and database	apps/api/ — REST, migrations, the doctor	C1
Triggers and keys	apps/api/ — webhooks, schedules, OAuth	C2
Google nodes	packages/nodes/google/	D1
Deployment and measurement	infra/ , benchmarks/ , docs/	D2

26 Proving the whole thing works, on the day

The morning of

Press Connect on every Google credential. In Testing mode a refresh token lasts seven days. It takes forty seconds, and it has ended more student demos than any bug.

Then fire the webhook once from a phone, on mobile data, and confirm a row appears.

The demonstration, in order

1 · Draw it live on an empty canvas. 2 · Change an operation and watch the form change. 3 · Show an expression preview. 4 · Fire it from a phone. 5 · Watch the nodes light up one at a time. 6 · A real email, a real row. 7 · Break it on purpose. 8 · Show the failed run in the executions list.

SHOW THIS	IT PROVES
A misspelled node name in a connection	validation happens before anything runs
<code>while (true) {}</code> in the Code node	the timeout fires, and the server still answers
A Sheets-only credential on the Gmail node	the scope sentence, not a bare 403
The same webhook twice with one <code>Idempotency-Key</code>	one execution, not two
Saving from two browser tabs	409, and nobody's work is lost
An expression with a typo	the panel warns instead of crashing

Every line above is a deliberate test, not an accident. Run all six before the demo and screenshot them. They are the evidence that the platform fails safely, which is most of what “robust” means to a marker.